

Operational support systems for the future

D Freestone and M Owen

The information services market, emerging from the convergence of telecommunications, information technology, entertainment and other industries, poses serious challenges for operational support systems. Historical ways of understanding and delivering operational support functions will no longer work. Business management functions will increasingly be a major commercial differentiator in the new market. This paper argues that these functions need to be re-defined as business services so that a market can develop in them. It explains a technical approach to defining and building these business services, which is being researched at BT Laboratories, and which uses configurable business objects and intelligent agents. This approach offers the possibility of managing the complexity that the explosion of new information services will bring, and also of opening up exciting new business possibilities.

1. Introduction

As other papers in this journal show, operational support systems (OSS) carry out a wide range of functions. This paper concentrates on business management functions — e.g. pricing and billing, provisioning, portfolio management, customer help — rather than network management or service control functions.

Business management functions will increasingly be a major differentiator in information systems markets. The organisation that can manage the deployment of new information services rapidly, and integrate them easily into a well-structured support system, will have commercial advantage over the organisation that has to handle each new information service separately and create customised support systems for it.

This paper argues that business management functions should be redefined as encapsulated business services so that a market can develop in them. It goes on to explain a technical approach to defining and building these services, which is being researched at BT Laboratories, using the technology of configurable business objects. It also describes how intelligent agents technology can help.

Section 2 of the paper discusses business management functions and explains why they are better considered as business services with commercial value. Section 3 discusses current systems which deliver operational support and explains why they have to change. The business objects approach is described in section 4, with configurable business objects discussed in section 5. Section 6 gives more detail about two prototype business objects that have been built at BT Laboratories and section 7 outlines some

roles for intelligent agents. Section 8 speculates on new commercial and technical opportunities that could open up if a market develops in business services.

2. Business services — key to the future

Operational support systems for the future have to support the new services and markets that are developing. A very significant part of this is the information services market, which stems from the convergence of several industries, principally telecommunications, information technology, entertainment and other information providers.

There is much debate about what the structure of the information services industry will be like in five or ten years time. Figure 1 illustrates a common view [1]. It suggests

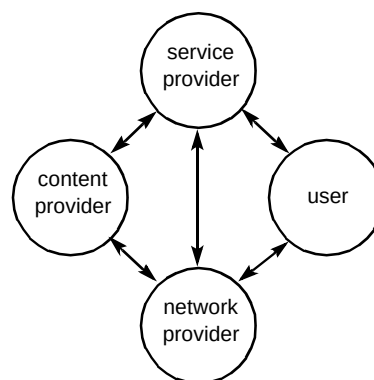


Fig 1 A common view of the future information services market.

there will be a thriving market of small niche organisations, divided into content providers, service providers and network providers, which work together to provide integrated services. An early example of this is the Internet,

where there are many different service providers and millions of content providers, all independent of the network providers.

Manageability of services will be a key market differentiator for providers. For example, for service providers, the quality of cross-product support, the ease of introducing new products and services, and of removing them, will all be direct contributors to profitability.

The notion of constructing and deploying a service, complete with both application functionality and support functions is often called 'total service creation' [2]. To achieve that, flexibility is needed in the support functions; they are likely therefore to be compartmentalised (encapsulated) and perhaps eventually commoditised. At the time of writing, some of the more obviously self-contained support functions are already being marketed independently, particularly billing (often by credit card companies) and customer help ('helpdesk'). Examples are customer service centres and helpdesks from IT Southern (part of Southern Water Group) [3] and from Siemens-Nixdorf Belgique-Luxembourg [4], billing, helpdesk, content management and network management from Digital Outsourcing Management Services [5], and billing from Carnegie-Mellon University [6].

Such encapsulated support functions are essentially business services and they share many of the characteristics of information services. It is reasonable to expect the market in business services to develop and extend to cover other support functions beyond billing and helpdesk. One organisation may specialise in a particular business service (e.g. billing), another on integrating applications and business services into total customer packages. There will be a particularly important role for organisations that can integrate support functions across product packages, customer communities and market segments.

To enable a market there needs to be some degree of standardisation, particularly for the interfaces between the different business services. Various implementations are then possible which can meet the demands of different volumes, different application technologies, different parameters, etc, but which all deliver the standard functionality. For example, content providers, service providers and network providers may use different implementations. At the top level, all three types of provider require pricing mechanisms, ordering mechanisms, fault tracking, etc, but the details of the business processes and the parameters involved may be different. Content providers may operate mainly through bulk contracts, network providers may price on the basis of bandwidth used or reserved, and service providers may charge by time or by transaction. Similarly, one charging and billing service may offer credit facilities, another may offer pay-as-you-go

options using advanced mechanisms like micro-payments [6, 7].

3. Why operational support systems need to change

The emerging information services market and the prospect of a dynamic market in business services presents serious challenges for traditional operational support systems.

Broadly, two different types of OSS are currently deployed — large strategic systems for volume core services [8–10], and quick tactical solutions for high-speed low-volume services (e.g. on-line information services). In both cases the OSS are separate systems (or combinations of systems) managed independently from the services they support, and often remote from them.

Both types of OSS are challenged by the new market contexts. Large-scale strategic systems generally have very long lead times and so cannot respond rapidly to new service requirements. On the other hand, short-term tactical solutions are typically isolated systems ('stove pipes') which do not interwork with each other or with the strategic systems, and so do not allow cross-service packages to be marketed. The two types of OSS have completely different design philosophies and it is difficult to migrate services from one type to the other. Figure 2 illustrates the two types of OSS and the challenges they face, plotted against the kinds of services they are deployed to support.

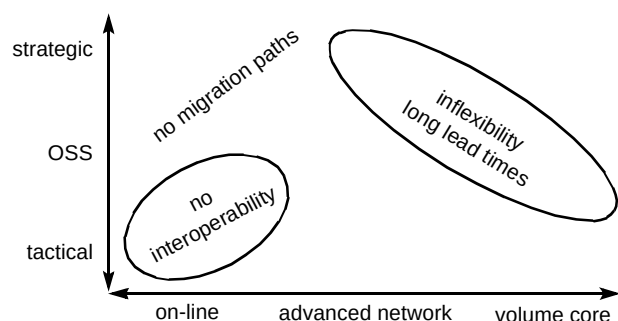


Fig 2 Challenges faced by current OSS.

A further issue is the execution cost of delivering the operational support functions. For example, for some information services like reading a single World Wide Web page, the cost of capturing and storing individual transaction records, and processing them to construct a bill, may significantly exceed the transaction price the market will bear. In such cases it may be necessary to tailor the support function around the commercial constraints. However, there are often other reasons why data records are kept, e.g. for marketing intelligence and future product planning. There is a complicated trade-off here between the benefits of storing data and the costs incurred. Part of the resolution of that is to have a good unit cost model of

individual OSS (i.e. business management) functions. The business services approach outlined in section 2 helps do that, by forcing the functions to be considered separately.

A new way of delivering OSS (i.e. business management) functions is needed which is compatible with the business services approach discussed in section 2. It needs to provide interoperability between different implementations, scalability, support for third party suppliers, and flexibility to provide many different variants for different services.

Operational support systems have already been modified to meet major changes in the products and services they support. Until recently, they tended to be closely coupled to individual product lines, leading to a proliferation of systems with limited interoperability. In that environment, each new product or service was required to interface with a heterogeneous group of back-end systems to fulfil business and product requirements. Particular OSS systems have required specific changes to overcome these difficulties [8–10].

Two current initiatives within BT are trying to provide an overall infrastructure for OSS, to provide functional coherence across products and services and to shield them from multiple OSS interfaces — the BT OSS architecture [11] and the three-tier architecture [12, 13].

This paper highlights the need for further step changes in the way OSS are conceived, designed and deployed; that need has been motivated and discussed in the earlier part of this paper. The following sections describe research under way at BT Laboratories to produce the next step change.

4. Business objects

To achieve encapsulation as discussed in section 2, the overall business management functionality (i.e. the part of operational support that handles business management) is partitioned into discrete business services. At the top level, the business services are coarse grained and Fig 3 shows a possible partitioning.

One approach to delivering flexible, interoperable business services for the future is to deploy a business object (in the sense of object technology) for each instance of a business service that is required. Some aspects of this build on suggestions from previous researchers, e.g. in the European RACE programme [14].

To carry out their functions, the business objects interact with each other and with the information services they are supporting. There can also be direct interaction between business objects and customers (e.g. to place an order or change a tariffing option), or between business objects and a business operations manager (e.g. to change prices or to

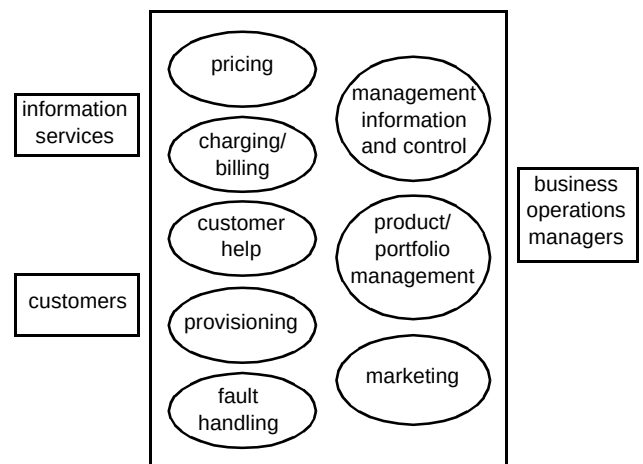


Fig 3 Business services.

deploy a new information service). Consequently several different types of interface need to be defined:

- between each pair of business objects,
- between the business objects and the generic information services being supported,
- for direct customer interaction,
- for a business operations manager.

In general, the interfaces need to be application programming interfaces (APIs).

The notion of standardised APIs is relatively novel for OSS functionality and it offers several benefits. Firstly, it allows different instantiations of individual business objects depending on the application requirements, but in such a way that the different instantiations can interoperate. In addition, it is possible to move from one instantiation to another (e.g. to increase capacity to cope with greater volumes) without affecting the functional interactions. Secondly, different instantiations can be used at different granularities of operation, e.g. one instantiation could be attached to an individual information service, a second instantiation to groups of information services, and a third instantiation to an entire business. Thirdly, standardised APIs allow information services from multiple service providers to be packaged together via common interfaces.

Similar approaches are being investigated by other organisations for related domains [15]. In particular, the Object Management Group (OMG), an industry consortium which promotes object technology for distributed systems, has initiated a call for standardised technology (including standard APIs) to handle business objects in its CORBA (Common Request Broker Architecture) environment [16, 17].

A major research issue with business objects is designing the best granularity of operation, as mentioned above. At one (highly distributed) extreme, functionality could be attached to each atomic information service, with functionality built up incrementally through a service composition hierarchy to provide the support functionality for the overall service package. At the other extreme, a centralised OSS server could be used. In between are many gradations. The options need to be assessed for flexibility, ease of use, engineering feasibility and cost-effectiveness. Probably different distribution regimes will be appropriate for different business services (e.g. pricing might be highly decentralised and charging/billing centralised).

5. Configurable business objects

The research within BT Laboratories (BTL) is going one step further than ordinary business objects. It is creating configurable business objects. These are business objects which are preprogrammed with many different options. The options are intended to handle most of the flexibility required to cover future information services and changes to information services already deployed. Objects are customised for individual information services (or packages of information services) by selecting options either at service creation time (e.g. to select order handling mechanism) or at run time (e.g. to set pricing policies). Options are selected by programming the interface to handle the parameters needed. It should be noted that this instantiation has to follow meta-rules to ensure that interoperability with other services still applies — and this area is an aspect of the research.

This approach builds on previous work which has successfully created and deployed a smaller scale business object (for VAT processing) with future-proofing options [18].

The development method used at BTL to produce configurable business objects starts by using a simplified, light-weight version of Jacobson's 'use case' analysis [19, 20]. Example 'use cases' are selected for analysis and their points of overlap form the basis of the object-based design. To give flexibility, two lines of defence are built into the design. Firstly, many options are precoded as parameters, adjustable by simple switches on the business operations managers' interface (see Fig 3). In particular, policy objects have been devised and used for issues where several possible policies can be defined in advance. Secondly, the encapsulation property of objects is used to decouple aspects of the design that are likely to be volatile from those judged likely to remain stable. This simplifies any future changes that are more radical.

The prototype configurable business object is used in a series of workshops involving domain experts who are invited to 'play' with it. This process has two aims. The first is to try and find design errors, often subtle effects from policy interactions. The second is to form judgements on what functionality should be included, and what excluded,

for flexibility purposes. The experts' views are paramount on judging the trade-off between flexibility and cost of initial effort.

Flexibility does not come free. It consumes creative energy and construction costs, and it also potentially makes the run-time object bigger than it might otherwise need to be (depending on how the instantiation works). There is a trade-off to be worked out between these costs and the benefits of being able to configure a business service very quickly and having very few integration tests to carry out at each service installation.

6. Pricing and charging — a worked example

Within the research programme at BTL, prototypes have been built for pricing, charging/billing and management information and control. They are designed to be compatible with internal major systems where this is relevant (e.g. the flexibility points in the multiservice billing system described in Crookes [21]). Distributed Smalltalk [22] has been chosen for the implementation technology, because it provides a fast prototyping environment with a standard interfacing mechanism (CORBA 2 [23]).

Figure 4 shows the structure of the pricing and charging/billing business objects, slightly simplified to highlight the important design principles. There are three main components — pricing, charging/billing and persistent storage, which is used by both business objects. Initially, the pricing object operates with one configured instance per information service (i.e. there are multiple instances of the object), but there is just one centralised instance of the persistent storage and one of the charging/billing object. This is an example of the 'granularity of operation' discussed in section 4.

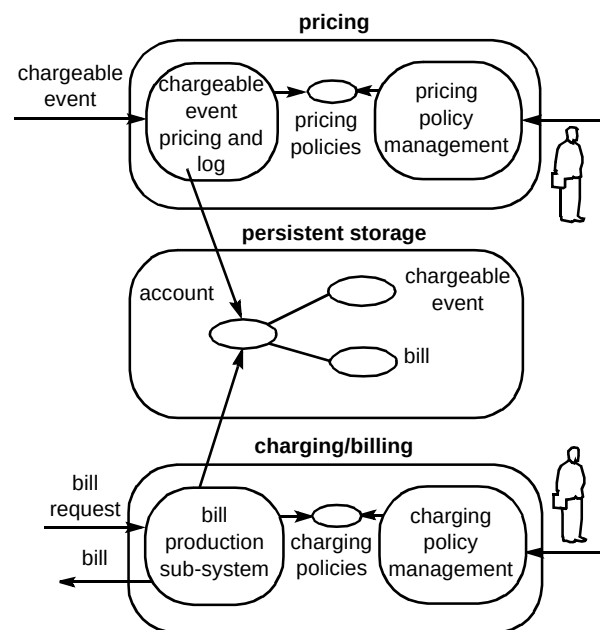


Fig 4 Pricing and charging/billing.

Operation of the business objects is triggered by external stimuli, indicated on the left of Fig 4. Information services generate streams of chargeable events. The choice of events or thresholds to count as chargeable events is made at service construction time, the same time that pricing policies are chosen. The chargeable events are sent to the pricing object, to be priced using the pricing policy currently in force for that service and that customer. Priced chargeable events are stored in the persistent storage.

The charging/billing object constructs a bill by extracting all the relevant (priced) chargeable events from the persistent storage and applying any 'charging policies' that apply. Charging policies are modifications to groups of chargeable events or an entire bill, e.g. volume discounts, penalty surcharges or affinity discounts. This business object is designed to provide after-the-event accounting and billing, but it can be configured to construct just statements of account if prepayment mechanisms are used, as discussed in section 2.

Within the persistent storage, 'account' is the primary record, and for each account records are kept of priced chargeable events and bills constructed. (In fact, there need to be several other subordinate records, e.g. customer details, but they are not relevant to the main features being discussed here.)

Key features of both business objects are policies and policy management. Pricing policy management and charging policy management are examples of the policy objects discussed in section 5. The management interfaces to allow switching between policy options are indicated by the people on the right of Fig 4, and they are examples of the business operations managers' role outlined in section 4.

To illustrate the kind of switched options that can be preprogrammed, Table 1 shows three of the pricing options built into the prototype pricing object. Each policy option has policy parameters stored in the policy object.

Table 1 Prototype pricing options.

Policy type	Policy parameters	Chargeable event parameters	Comments
Subscription	price		subscription frequency is handled by the information service itself
By content	unit_price descriptor	number_of_units	descriptor is the text to go on bill, e.g. 'kb storage used', 'messages read'
By duration	interval_price interval_length	start_time end_time	

The aim with policies is to provide enough options (suitably parameterised) to cater for a wide range of information services. However, as it is impossible to predict all policy requirements that will be needed in the future, it is important to offer a programmable interface as well, to allow the introduction of completely new policies. The 'second line of defence', the use of object encapsulation to isolate potentially volatile aspects of the design, mentioned in section 5, helps this. Connecting the programmable interface to a specific sub-object isolates the programmatic impact of a new policy, and provides the constraints to ensure interworking with other business objects and other information services.

One internal trial, with an advanced messaging service, has already been completed using the pricing and charging/billing prototypes. This gave useful feedback for both the messaging service and the business services, neither of which would have been possible without a joint trial. Further trials are planned, including larger scale trials with several services involved.

7. Intelligent agents

Another technology which will help deliver flexible, interoperable business services is that of intelligent agents, which is also being researched at BTL.

Intelligent agents originate from the field of distributed artificial intelligence (DAI). Research in this area is concerned with understanding and modelling action and knowledge in collaborative enterprises [24]. Autonomous intelligent systems, known as agents, pursue local or global goals, co-ordinate actions, share knowledge and resolve conflicts. By providing agents with both reactive and goal-directed behaviour, they are given high-level problem-solving skills enabling them to act intelligently on behalf of the entities they represent within the problem domain. A multiagent system can be defined as 'a loosely coupled network of problem solvers that work together to solve problems that are beyond their individual capabilities' [25]. DAI techniques have been successfully applied to a number of telecommunications domains [26, 27]. Business services operate in an open, heterogeneous and decentralised management environment in which both control and data are distributed. Agent technology therefore lends itself to this domain.

Agents have several potential uses. One of them is to guide humans through processes which are constantly being updated. The benefit of this is minimising the training needed. For example, consider a helpdesk business service.

Information service providers face a common problem in providing accurate guidance to helpdesk staff performing operational business processes. There is a continual need to update them to take account of new services, incremental

changes to existing services and cross-service support. If the underlying knowledge about business processes is encapsulated as sets of plans, then intelligent agents can be used to guide customer-facing people through complex sequences of actions with minimal effort and cost.

For this kind of use, the agent architecture commonly referred to as BDI (beliefs, desires and intentions) [28] is appropriate. (This is one of a number of agent architectures which exist and which use different ways of putting intelligence into the agents.) In BDI, an agent's state is modelled by its beliefs. In the help desk service, an agent's beliefs may include information concerning a customer, e.g. a customer's contract with BT. An agent's objectives can be expressed in terms of achieving an appropriate business process. These objectives are represented by an agent's goals, or desires. The knowledge concerning how to perform a particular business process is captured in the form of pre-set plans which represent a sequence of actions to be performed. These actions may include requesting information from the customer, extracting data from other business services, e.g. billing or fault handling, and presenting information back to the customer. Plans may be assigned varying priorities describing their importance in relation to other plans. A goal is invoked by a combination of a context (determined by the current state of the agent, e.g. the information held about a particular customer) and a triggering event, such as a fault report. An agent's intention refers to the particular goal and associated plan which has been invoked. Agents can prioritise the tasks to be completed based on the priority level assigned to a plan. In this way, agents are provided with both reactive and goal-directed behaviour, enabling the agents to act intelligently.

Another benefit from using agents is the help they can give to business objects to deliver new business processes even when there are legacy systems involved. Legacy systems are strategic systems, critical to the business mission, which have become difficult to maintain following many changes to their requirements. Many OSS in telecommunications organisations quickly become legacy systems because of the succession of rapidly changing requirements which are inevitable in a fast-moving industry.

Depending on the number of interfaces, the interconnectedness of functionality and the performance requirements, legacy systems can sometimes be encapsulated using wrapper code and accessed by agents. This approach facilitates the tailoring of business services for individual applications and customers.

8. New opportunities from OSS

Building operational support functions from discrete business services and using intelligent agents opens up a number of new commercial and technical opportunities.

With a complete set of prototyped business services, it is possible to hold total service workshops, on similar lines to the flexibility workshops described in section 5. Experiments on the total operational characteristics of a new service can be undertaken in advance of launch, to get a better idea of how the package will appear, including pricing, provisioning mechanisms, helpdesk support, fault handling, etc.

With run-time switched policies in business services, it will be possible to tailor the data stored to the policy chosen. This could enable dynamic resolution of the trade-off between storage costs and the benefits from stored records. Therefore, if the relative weights changed, then so could the storage policy. It is not necessary always to store the maximum amount of data just in case it is needed.

With a thriving market in business services, a service provider can shop around for business services to attach to information services. Different business services may be chosen from different suppliers, depending on price, platform, performance characteristics, etc, and integrated either by the service provider or by an integration service. The ability to manage such diversity will be a key market differentiator. Intelligent agents could construct customised business service packages based on customer-supplied goals (e.g. total performance, customer interfaces, maximum grade of support for a given price), and the agents could keep a watch on the market so the packages could be dynamically reconfigured.

With business service policies switchable at run time, it will be possible to monitor macro business parameters in near real time (e.g. revenues, volumes and types of customer queries). Then policies could be modified dynamically to impact upon business performance, e.g. by changing prices (running a 'happy hour') or altering standard help information (updating frequently asked questions lists). The decision and control issues involved in dynamic management of business parameters will need careful consideration. Developments in distributed problem solving from the field of distributed artificial intelligence [29, 30] may need to be involved.

9. Conclusions

The convergence of telecommunications, information technology and entertainment industries is creating an information services market. The flexibility and rapid deployment needed in this new market mean that historical ways of delivering operational support functions will no longer work. Operational support systems must therefore change. Many incremental changes have already been implemented by telecommunications operators to respond to new services and new trading conditions, and significant infrastructure upgrades are planned.

However, this paper has argued that those changes are not enough. A radical re-think is needed, which offers more hope of managing the complexity which the explosion of new services will bring. Increasingly, operational support will be a key market differentiator for players in the information services market — it is already the case that providing operational support functions can take longer than providing applications.

All serious players in the market are having to grapple with these problems. This paper has advocated thinking of the issues in terms of a market in business services and dynamic management of business parameters. It has also discussed research approaches being investigated at BT Laboratories. On such research hangs the future of the industry.

References

- 1 Greenop D: 'Shaping the European information society', British Telecommunications Engineering J, 14, Part 3 (October 1995).
- 2 Turner G D: 'Service creation', BT Technol J, 13, No 2, pp 80—86 (April 1995).
- 3 <http://www.itsouthern.co.uk/htdocs/mansrv7.htm>
- 4 <http://www.sni.be/fr/offer/services/its/operations/operweb.htm>
- 5 <http://www.digital.com/.i/info/mcs/outourcing/index.htm>
- 6 Sirbu M and Tygar J D: 'NetBill: an Internet commerce system optimised for network delivered services', IEEE CompCon Conference (1995).
- 7 Rivest R L and Shamir A: 'Payword and micromint: two simple micropayment schemes', to appear: <http://theory.lcs.mit.edu/~rivest/RivestShamir-mpay.ps>
- 8 White P: 'A service system for BT's major customers', BT Technol J, 15, No 1, pp 81—86 (January 1997).
- 9 Harrison P F: 'Customer Service System — past, present and future', BT Technol J, 15, No 1, pp 29—45 (January 1997).
- 10 Chandler J: 'Management of special services — designing for a changing world', BT Technol J, 15, No 1, pp 87—97 (January 1997).
- 11 Furley N: 'The BT operational support systems architecture', BT Technol J, 15, No 1, pp 13—21 (January 1997).
- 12 Mathieson G: 'Customer handling intermediate server — an architecture-led project', BT Technol J, 15, No 1, pp 114—134 (January 1997).
- 13 Winton N: 'Management for Distributed Computing Environment-based applications', BT Technol J, 15, No 1, pp 160—166 (January 1997).
- 14 Strick and Meinkohn: 'Specifying Pan-European management systems', in Kugler, Mullery, Niebert (Eds): 'Towards a pan-European telecommunications service infrastructure', Springer-Verlag (1994).
- 15 Sutherland J (Ed): 'Business object design and implementation workshop', OOPSLA '95 ACM Press (1995).
- 16 <http://www.omg.org/pr96/bomrfp.htm>
- 17 <http://www.omg.org/cftfrfp4.htm>
- 18 Freestone D and Wezeman C: 'An object lesson in changing legacy systems', in O'Callaghan A J (Ed): 'Practical experiences of object technology', Stanley Thornes (1996).
- 19 Jacobson I: 'Object-oriented software engineering — a use case driven approach', Addison Wesley (1992).
- 20 Jacobson I: 'The object advantage; business process re-engineering with object technology', Addison-Wesley (1995).
- 21 Crookes J: 'Multiservice billing system — a platform for the future', BT Technol J, 15, No 1, pp 98—113 (January 1997).
- 22 Hewlett Packard: 'Distributed Smalltalk Programmer's Reference Guide: Edition 5', (B4047-90038) (1995).
- 23 Siegel J: 'CORBA fundamentals and programming', Wiley (1996).
- 24 Gasser L: 'Social conceptions of knowledge and action: DAI foundations and open systems semantics', Artificial Intelligence, 47, pp 107—138 (1991).
- 25 Durfee E H, Lesser V R and Corkill D D: 'Trends in co-operative distributed problem solving', IEEE Trans Knowl Data Eng, 11, No 1, pp 63—83 (1989).
- 26 Liebowitz J and Prerau D S: 'Worldwide intelligent systems', IOS Press (1995).
- 27 Smith R, Azarmi N and Nwana H S (Eds): 'Intelligent software systems', BT Technol J, 14, No 4 (special issue) (October 1996).
- 28 O'Hare G M P and Jennings N: 'Foundations of distributed artificial intelligence', Wiley (1996).
- 29 Corkill D D and Lesser V R: 'The use of meta-level control for co-ordination in a distributed problem solving network', Proceedings of the 1983 International Joint Conference on Artificial Intelligence, pp 748—756 (1983).
- 30 Findler N V and Gao J: 'Dynamic hierarchical control for distributed problem solving', Data and Knowledge Engineering, 2, pp 285—301 (1987).



David Freestone holds degrees in Mathematics from Cambridge University, and Computation from Oxford University. He is a member of the BCS and a Chartered Engineer. Since joining BT Laboratories in 1980, he has worked on programming support environments, formal methods for communications protocols and advanced techniques for requirements engineering. Currently he leads a team researching how to provide rapid and flexible operational support for future services.



Martin Owen graduated from Loughborough University in 1975 with an MSc in Mathematics and joined the teaching profession. He joined BT Laboratories in 1991 after receiving an MSc in Computer Studies from the University of Essex. He initially worked in requirements capture and analysis for network management systems. He is currently working in the Systems and Software Strategy unit, developing prototypes demonstrating the use of agent technology in service management.